

Non-transient `constexpr` allocation

Document #: P2670R0
Date: 2022-10-15
Project: Programming Language C++
Audience: EWG
Reply-to: Barry Revzin
<barry.revzin@gmail.com>

Contents

- [1 Introduction](#)
- [2 The Problem Case](#)
 - [2.1 Mutable Allocation](#)
 - [2.2 Mutable Reads During Constant Destruction](#)
- [3 Proposed Solutions](#)
 - [3.1 `std::mark\_immutable\_if constexpr`](#)
 - [3.2 `T constexpr\*`](#)
 - [3.3 Disposition](#)
 - [3.4 Alternatives](#)
- [4 Proposal](#)
- [5 References](#)

§ 1 Introduction

For C++20, [P0784R7] introduced the notion of transient `constexpr` allocation. That is, we can allocate during compile time - but only as long as the allocation is completely cleaned up by the end of the evaluation. With that, we can do this:

```
constexpr auto f() -> int {  
    std::vector<int> v = {1, 2, 3};  
    return v.size();  
}  
static_assert(f() == 3);
```

But not yet this:

```
constexpr std::vector<int> v = {1, 2, 3};  
static_assert(v.size() == 3);
```

Because `v`'s allocation persists, that is not yet allowed.

§ 2 The Problem Case

The problem with persistent `constexpr` allocation can be demonstrated with this example:

```

1 // simplified version of unique_ptr
2 template <class T>
3 class unique_ptr {
4     T* ptr;
5 public:
6     explicit constexpr unique_ptr(T* p) : ptr(p) { }
7     constexpr ~unique_ptr() { delete ptr; }
8
9     // unique_ptr is shallow-const
10    constexpr auto operator*() const -> T& { return *ptr; }
11    constexpr auto operator->() const -> T* { return ptr; }
12
13    constexpr void reset(T* p) {
14        delete ptr;
15        ptr = p;
16    }
17 };
18
19 constexpr unique_ptr<unique_ptr<int>> ppi(new unique_ptr<int>(new
    int(1)));
20
21 int main() {
22     // this call would be a compile error, because ppi is a const
23     // object, so we cannot call reset() on it
24     // ppi.reset(new unique_ptr<int>(new int(2)));
25
26     // but this call would compile fine
27     ppi->reset(new int(3));
28 }

```

In order for constexpr allocation to persist to runtime, we first need to ensure that all of the allocation is cleaned up properly. We can check that our allocation (stored in `ppi.ptr`) is properly deallocated in its destructor (which it is). And so on, recursively - so `ppi.ptr->ptr` is properly cleaned up in `*(ppi.ptr)`'s destructor.

Once we verify that constexpr destruction properly deallocates all of the memory, no actual destructor is run during runtime. In order for this to be valid, it is important that the destruction that would happen at run-time is actually the same as the destruction that was synthesized during compile-time.

In the above example though, that is not the case.

`ppi` is a `unique_ptr<unique_ptr<int>> const`, so its member `ppi.ptr` is a `unique_ptr<int>* const`. The pointer itself is `const` (top-level), which means that we cannot change what pointer it owns. This means that destroying `ppi` at runtime would definitely `delete` the same pointer that it was instantiated with at compile time (short of `const_cast` shenanigans, which we can't do much about and are clearly UB anyway). So far so good.

But `*ppi` gives me a *mutable* `unique_ptr<int>`, which means I can change it. In particular, while `ppi.reset(~)` would be invalid, `(*ppi).reset(~)` would compile fine. Our tentative evaluation of the destructor of `*(ppi.ptr)` would try to delete the pointer which was initialized with `new int(1)`, but now it would point to `new int(3)`. That's something different, which means that our synthetic destructor evaluation was meaningless.

More to the point - the `delete ptr;` call (on line 14) would end up attempting to delete memory that wasn't allocated at runtime - it was allocated at compile time and promoted to static storage. That's going to fail at runtime. This is highly problematic: the above use looks like perfectly valid C++ code, and we got into this

problem without any `const_cast` shenanigans or really even doing anything especially weird. It's not like we're digging ourselves a hole, it's more that we just took a step and fell.

§ 2.1 Mutable Allocation

The problem isn't that the allocation is mutable. It's much more subtle than that. There are no problems allowing this:

```
constexpr unique_ptr<int> pi(new int(4));
```

While the data `pi` points to is mutable, it can be mutated at runtime without any issue. The important part of this is that the allocation is pointed to by a const pointer (`pi.ptr` here is an `int* const`), so there is no way to change *which* pointer is the allocation. Just what its data is.

The expectation would be that the above program would basically be translated into something like:

```
int __backing_storage = 4;
constexpr int* pi = &__backing_storage;
```

That is perfectly valid code today - it's the value of the pointer that is a constant, not what it is pointing to.

§ 2.2 Mutable Reads During Constant Destruction

The problem is specifically when a *mutable* object is *read* during *constant destruction*.

In the previous section, `pi.ptr` (an `int* const`) is read during constant destruction, but it is not mutable. `* (pi.ptr)` is mutable (an `int`), but wasn't read.

In the original example, `ppi.ptr` (a `unique_ptr<int>* const`) is read during constant destruction, but it is not mutable. But `ppi.ptr->ptr` is mutable (it's just an `int*`) and was read during `*(ppi.ptr)`'s constant destruction, which is the problem.

While `constexpr unique_ptr<unique_ptr<T>>` might not seem like a particularly interesting example, consider instead the case of `constexpr vector<vector<T>>` (or, generally speaking, any kind of nested containers - like a `vector<string>`). Both `unique_ptr<T>` and `vector<T>` have a `T*` member, so they're structurally similar - it's just that `unique_ptr<T>` has a shallow const API while `vector<T>` has a deep const API.

But this is purely a convention. How can the compiler distinguish between the two? How can we disallow the bad `T*` case (`unique_ptr<unique_ptr<T>>` or `unique_ptr<string>` or `unique_ptr<vector<T>>`) while allowing the good `T*` case (`vector<vector<T>>` or `vector<string>`)?

§ 3 Proposed Solutions

There have been two proposed solutions to this problem, both of which had the goal of rejecting the nested `unique_ptr` case:

```
constexpr unique_ptr<int> pi(new int(1)); // ok
constexpr unique_ptr<unique_ptr<int>> ppi(new unique_ptr<int>(new int(2))); //
error
```

While allowing the vector cases:

```
constexpr std::vector<int> vi = {1, 2, 3}; // ok
constexpr std::vector<std::vector<int>> vvi = {{1, 2}, {3, 4}, {5, 6}}; // ok
constexpr std::vector<std::string> vs = {"this", "should", "work"}; // ok
```

§ 3.1 `std::mark_immutable_if_constexpr`

The initial design in [P0784R7], before it was removed, was a new function

`std::mark_immutable_if_constexpr(p)` that would mark an allocation as being immutable. That is, rather than rejecting any object that is mutable and read during constant destruction, the new rule would be any object read during constant destruction must be either:

- constant, or
- marked as immutable

`std::vector<T>` would mark its allocation as immutable (e.g. at the end of its constructor), but `std::unique_ptr<T>` would not (because it's only shallow const).

That way:

```
// ok: this is fine (no mutable object is read during constant destruction)
constexpr std::unique_ptr<int> a(new int(1));

// error: allocation isn't marked immutable, but is read as mutable during
//        constant destruction
constexpr std::unique_ptr<std::unique_ptr<int>> b(new std::unique_ptr<int>(new
    int(2)));

// ok: allocation isn't marked immutable, but is read as constant
constexpr std::unique_ptr<std::unique_ptr<int> const> c(new std::unique_ptr<int>
    (new int(3)));

// ok: allocation isn't read as mutable
constexpr std::vector<int> v = {3, 4, 5};

// ok: allocation is read as mutable but is marked immutable
constexpr std::vector<std::vector<int>> vv = {{6}, {7}, {8}};
```

In this way, `std::mark_immutable_as_constexpr(p)` is basically saying: I promise my type is actually properly deep-`const`. Note that there's no real enforcement mechanism here - if the user did mark their `unique_ptr` member immutable, then the original example in this paper would compile fine (and fail at runtime). It's just a promise to the compiler that your type is actually deep-`const`, it's up to the user to properly ensure that it is.

§ 3.2 `T propconst*`

[P1974R0] proposed something different: a new qualifier that would propagate `const`-ness off the object through the pointer. A persistent `constexpr` allocation would thus have to have only `const` access, after adjusting through `propconst`.

`std::vector<T>` would change to have members of type `T propconst*` instead of `T*`, so that a `constexpr std::vector<int>` would effectively have members of type `int const*`, while `std::unique_ptr<T>` would

remain unchanged, still having a `T*`. That way:

```
// ok: this is fine (no mutable object is read during constant destruction)
constexpr std::unique_ptr<int> a(new int(1));

// error: b's pointer member has mutable access to a std::unique_ptr<int>
// which is a mutable read during constant destruction
constexpr std::unique_ptr<std::unique_ptr<int>> b(new std::unique_ptr<int>(new
    int(2)));

// ok: each allocation is read as constant
constexpr std::unique_ptr<std::unique_ptr<int> const> c(new std::unique_ptr<int>
    (new int(3)));

// ok: allocation isn't read as mutable
constexpr std::vector<int> v = {3, 4, 5};

// ok: allocation isn't read as mutable, because the member is now a vector<int>
// propconst* rather
// than a vector<int>*, so behaves as if it were a vector<int> const*
constexpr std::vector<std::vector<int>> vv = {{6}, {7}, {8}};
```

The distinction between the two proposals for a simple vector implementation:

mark_immutable_if_constexpr	T propconst*
<pre>template <typename T> class vector { T* ptr_; size_t size_; size_t capacity_; public: constexpr vector(std::initializer_list<T> elems) { ptr_ = new T[xs.size()]; size_ = xs.size(); capacity_ = xs.size(); std::copy(xs.begin(), xs.end(), ptr_); std::mark_immutable_if_constexpr(ptr_); // <== } constexpr ~vector() { delete [] ptr_; } }</pre>	<pre>template <typename T> class vector { T propconst* ptr_; // <== size_t size_; size_t capacity_; public: constexpr vector(std::initializer_list<T> elems) { ptr_ = new T[xs.size()]; size_ = xs.size(); capacity_ = xs.size(); std::copy(xs.begin(), xs.end(), ptr_); } constexpr ~vector() { delete [] ptr_; } }</pre>

On the left, we have to mark the result of every allocation immutable. On the right, we have to declare every member that refers to an allocation with this new qualifier.

§ 3.3 Disposition

Both of these proposals have similar shape, in that they attempt to reject the same bad thing (e.g. `constexpr std::unique_ptr<std::unique_ptr<int>>`) by way of having to add annotations to the good thing (allowing `constexpr std::vector<std::vector<T>>` to work, by either annotating the pointers or the allocation). Same end goal of which sets of programs would be allowed.

Importantly, both require types opt-in, somehow, to persistent allocation. `T propconst*` is more sound, in that the compiler *ensures* that there is no mutable persistent allocation possible, while `std::mark_immutable_if_constexpr(p)` is simply a promise that the user did their due diligence and properly made their type deep-`const`.

What I mean is that, using `propconst`, this error won't compile - we can't accidentally expose mutable access (not without `const_cast`):

```
template <typename T>
class bad_vector_pc {
    T propconst* data_;
    int size_;
    int capacity_;

public:
    // oops, int& instead of int const&
    constexpr auto operator[](int idx) const -> T& {
        // error: data_[idx] is an T const& here
        return data_[idx];
    }
};
```

But this would work just fine (until it explodes at runtime):

```
template <typename T>
class bad_vector_miic {
    T* data_;
    int size_;
    int capacity_;

public:
    constexpr bad_vector_miic(std::initializer_list<int> xs) {
        data_ = new T[xs.size()];
        std::copy(xs.begin(), xs.end(), data_);
        size_ = xs.size();
        capacity = xs.size();

        std::mark_immutable_if_constexpr(data_);
    }

    constexpr ~bad_vector_miic() {
        delete [] data_;
    }

    // oops, T& instead of T const&
    constexpr auto operator[](int idx) const -> T& {
        // this still compiles though
        return data_[idx];
    }
};
```

```
constexpr bad_vector_miic<bad_vector_miic<int>> v = {{1}, {2}, {3}};

int main() {
    v[0] = {4, 5}; // ok: compiles?
}
```

However, `τ propconst*` has the downside that it's fairly pervasive throughout the language - where a significant amount of library machinery would have to account for it somehow. Whereas, `std::mark_immutable_if_constexpr(p)` is extremely limited. No language machinery needs to consider its existence, and the only library machinery that does are those deep-const types that perform allocation that would have to use it, in only a few places.

On the other hand, `std::mark_immutable_if_constexpr` has the problem that users might not understand where and why to use it and, just as importantly, when *not* to use it. If some marks an allocation immutable on a shallow-const type, that more or less defeats the entire purpose of the exercise, since now they've just allowed themselves to do exactly the thing that wanted to avoid allowing.

§ 3.4 Alternatives

The `propconst` proposal adds annotation to the members. The `mark_immutable_if_constexpr` proposal adds annotation to the allocations. The only real other place to add an annotation to would be the class:

```
template <typename T>
struct vector propconst { ... };
```

It is possible that this could be made to have the same effect as the `propconst` member annotations without having to have such a large effect on the type system. On the one hand, this means that we can't have a type like `unique_ptr<int propconst>` or `observer_ptr<int propconst>`. On the other hand, it means that we don't have to add further language complexity to make such things work (see sections 7 and 8 from [\[P1974R0\]](#)). Perhaps this is a reasonable compromise?

An entirely different approach would be to eschew annotations altogether. That is - just allow all of these examples to compile, and make clear that it's undefined behavior to mutate persistent `constexpr` allocations (the allocation itself, not what's written in it) if they're read by the elided destructor. One downside of `std::mark_immutable_if_constexpr(p)` is that users might just eagerly add it to all allocations, even if they shouldn't - and if such a thing happens, then why even bother adding such a function? It's hard to say how often such a facility would be misused - and at least it's not overly difficult to provide good guidance for exactly when one should use it: on types that own allocations such that a constant object only provides constant access through that allocation. Perhaps a longer name like `std::allocation_is_deep_constant(p)` might convey this better, or `std::i_solemnly_swear_that_i_am_up_to_no_mutation(p)`. But also, having a facility such as `std::mark_immutable_if_constexpr(p)` gives users of correctly-written libraries (like `std` and `boost`) protection against accidentally writing `constexpr unique_ptr<string>` or `constexpr unique_ptr<vector<T>>`.

I'm not sure that either of these alternatives are better than the two we already have on the table.

§ 4 Proposal

Between `τ propconst*` and `std::mark_immutable_if_constexpr(p)`, the former provides a sound solution to the problem with neither false positives nor false negatives. The latter is unsafe and, like Rust's `unsafe`, needs

to be carefully reviewed, and can easily provide false negatives (accepted code that really should've been rejected). But, like Rust's `unsafe`, it should only appear in a very small number of places anyway, and making sure those uses are correct doesn't seem like an outrageous burden. After all, of all the types that own an allocation - there are probably way more containers (deep const) than smart pointers (shallow const),¹ so a random use is more likely to be correct than not.

Given the clear need for persistent `constexpr` allocation, we need to solve this problem one way or another. I think the right way to do that is `std::mark_immutable_if_constexpr(p)`, except probably renamed in such a way as to clarify the proper intent of its use.

Lastly, adopting `std::mark_immutable_if_constexpr(p)` would not preclude against adding `propconst` later on if we get better implementation experience with it.

§ 5 References

[P0784R7] Daveed Vandevoorde, Peter Dimov, Louis Dionne, Nina Ranns, Richard Smith, Daveed Vandevoorde. 2019-07-22. More `constexpr` containers.
<https://wg21.link/p0784r7>

[P1974R0] Jeff Snyder, Louis Dionne, Daveed Vandevoorde. 2020-05-15. Non-transient `constexpr` allocation using `propconst`.
<https://wg21.link/p1974r0>

1. This might suggest that we should mark shallow const allocations as shallow const, rather than marking deep const allocations as deep const. The problem is that this requires users to take active action to reject bad code, rather than active action to allow good code – which makes it more likely that the bad code will persist. On the other hand, how many smart pointers are there really? ↩